Programmation parallèle

Examen machine OS202 2026

Parallélisation d'une simulation de galaxie

Auteur : *CHACÓN GÓMEZ José Daniel*

École Nationale des techniques avancées
Mars 2026

Table des matières

1 Introduction **3**
1.1 Signification des étapes . . . 3

2 Description du problème **3**

3 Question préliminaire sur $N_k = 1$ **4**

4 Mesure initiale : affichage vs calcul **4**
4.1 Définition exacte des temps mesurés . . . 4
4.2 Mesures observées . . . 5

5 Étape 1 : parallélisation avec Numba **5**
5.1 Objectif . . . 5
5.2 Changements clés d'implémentation . . . 5
5.3 Principes de parallélisation retenus . . . 6
5.4 Benchmark sans affichage . . . 6

6 Méthodologie expérimentale de l'étape 1 **6**
6.1 Environnement expérimental . . . 6

7 Résultats de l'étape 1 **7**
7.1 Benchmark compute-only . . . 7
7.2 Graphiques du benchmark compute-only . . . 7
7.3 Benchmark interactif . . . 7

8 Discussion de l'étape 1 avec la loi d'Amdahl **8**

9 Étape 2 : séparation affichage / calcul avec MPI **9**
9.1 Objectif . . . 9
9.2 Changements clés d'implémentation . . . 9
9.3 Pourquoi exactement deux processus . . . 9
9.4 Protocole de communication . . . 9

10 Résultats de l'étape 2 **10**
10.1 Mode interactif . . . 10
10.2 Benchmark MPI hors affichage . . . 10
10.3 Lecture directe du benchmark . . . 10
10.4 Graphiques du benchmark . . . 11

11 Comparaison entre l'étape 1 et l'étape 2 **11**
11.1 Comparaison interactive 11

12 Étape 3 : calcul MPI complet avec cellules fantômes **12**
12.1 Objectif 12
12.2 Changements clés d'implémentation 12
12.3 Topologie retenue 12
12.4 Rôle des cellules fantômes et des réductions globales 13

13 Résultats de l'étape 3 **13**
13.1 Campagne de mesures 13
13.2 Table 2D des temps bout-en-bout 13
13.3 Détails sur les configurations stables 13
13.4 Graphiques 14

14 Déséquilibre de charge et splitting adaptatif **15**

15 Conclusion intermédiaire **15**

1 Introduction

Ce document présente l'avancement de l'examen machine de programmation parallèle consacré à la simulation d'une galaxie. Le programme considère un trou noir central massif et un grand nombre d'étoiles. À chaque pas de temps, il calcule les accélérations gravitationnelles, puis met à jour les positions et les vitesses au moyen d'un schéma de Verlet avant d'afficher la scène en trois dimensions.

Le travail est organisé par étapes afin de conserver une version exécutable de chaque implémentation intermédiaire. Le mot « étape » désigne ici une version précise du projet, avec un objectif technique bien délimité.

1.1 Signification des étapes

- **Étape 00 – baseline :** version de référence, séquentielle, utilisée pour comprendre le code, vérifier le comportement de la simulation et mesurer les temps initiaux.
- **Étape 01 – Numba avec threads :** version *shared memory*, où l'on parallélise seulement les parties sûres du calcul avec `Numba` et `prange`.
- **Étape 02 – séparation affichage / calcul :** version MPI avec deux processus, un pour l'affichage et un pour le calcul, afin d'isoler clairement le rendu graphique du noyau numérique.
- **Étape 03 – MPI complet sur la grille :** version en mémoire distribuée, où plusieurs processus se partagent la grille de calcul, échangent des cellules fantômes et reconstruisent l'état global pour l'affichage.

Ce rapport couvre désormais les étapes 01, 02 et 03. L'idée générale est simple : on commence par paralléliser localement le calcul avec Numba, puis on sépare affichage et calcul avec MPI, et enfin on distribue réellement le calcul sur plusieurs workers avec décomposition de domaine et cellules fantômes.

2 Description du problème

Le calcul direct des interactions gravitationnelles entre toutes les étoiles serait en $O(N^2)$. Pour réduire ce coût, le programme utilise une grille cartésienne dans le plan principal de la galaxie. Les interactions proches sont calculées de manière détaillée avec les cellules voisines, tandis que les interactions lointaines sont approchées via la masse totale et le centre de masse des cellules éloignées.

En pratique, chaque itération suit le schéma suivant :

1. placer les étoiles dans les cellules de la grille ;
2. calculer les masses totales et centres de masse des cellules ;
3. calculer les accélérations gravitationnelles ;
4. mettre à jour les positions et vitesses ;
5. afficher la scène.

L'enjeu de l'étape 1 est d'identifier les boucles sûres à paralléliser avec Numba, c'est-à-dire celles où chaque itération écrit dans une zone mémoire indépendante.

3 Question préliminaire sur $N_k = 1$

Le sujet demande d'expliquer pourquoi il n'est pas intéressant de choisir une valeur $N_k > 1$. La raison principale est géométrique : la galaxie simulée est essentiellement aplatie dans le plan Oxy, avec une faible épaisseur selon Oz. Découper davantage l'axe z ne change donc presque pas la qualité de l'approximation gravitationnelle, mais augmente le nombre de cellules, le coût de gestion de la grille et, plus tard, le volume de communication MPI.

> **Conclusion préliminaire.** Pour cette simulation, prendre $N_k = 1$ est un choix cohérent : il respecte la géométrie du problème et évite un coût algorithmique supplémentaire peu rentable.

4 Mesure initiale : affichage vs calcul

Avant toute parallélisation, il faut distinguer le coût du rendu graphique du coût du calcul physique. Le visualiseur affiche déjà deux temps :

- `Render time`
- `Update time`

Cette séparation est importante, car seule la partie calcul est directement concernée par l'étape Numba. Si l'affichage reste significatif, la loi d'Amdahl limitera le speed-up global observé dans la simulation interactive.

4.1 Définition exacte des temps mesurés

Les deux temps affichés dans la boucle interactive ne correspondent pas à un profilage fin au sens d'un outil dédié ; ils représentent une découpe simple de la boucle principale du visualiseur.

Dans `stages/01_numba_threads/visualizer3d.py`, trois instants sont relevés à chaque itération :

- t_1 : fin de l'itération précédente ;
- t_3 : juste après l'appel à `_render()` ;
- t_2 : juste après `update_points(updater(dt))`.

On obtient alors :

$$\texttt{Render time} = t_3 - t_1 \qquad \text{et} \qquad \texttt{Update time} = t_2 - t_3$$

Autrement dit :

- `Render time` inclut la gestion des événements SDL du début de boucle, le rendu OpenGL du frame courant et l'échange de buffers ;
- `Update time` inclut l'appel au calcul physique `updater(dt)` ainsi que la copie des nouvelles positions dans le visualiseur via `update_points` ;
- `Update time` n'inclut ni la gestion d'événements ni le rendu ;
- la première itération est fortement perturbée par l'initialisation et, dans le cas Numba, par la compilation JIT.

En conséquence, ces deux temps donnent une bonne vision macroscopique de la boucle interactive, mais ils ne constituent pas un profilage détaillé instruction par instruction.

4.2 Mesures observées

Les premières mesures ont été relevées sur la version de référence `nbodies_grid_numba.py`, avec `data/galaxy_1000`, $dt = 0.0015$ et une grille $15 \times 15 \times 1$. Après l'itération initiale de compilation JIT, les valeurs stables observées sont d'environ 5 ms pour le rendu et 52 ms pour la mise à jour physique.

Mesure initiale	Valeur
Temps de rendu	≈ 5 ms
Temps de mise à jour	≈ 52 ms
Partie dominante	calcul des trajectoires

TABLE 1 – Mesure initiale pour comparer affichage et calcul

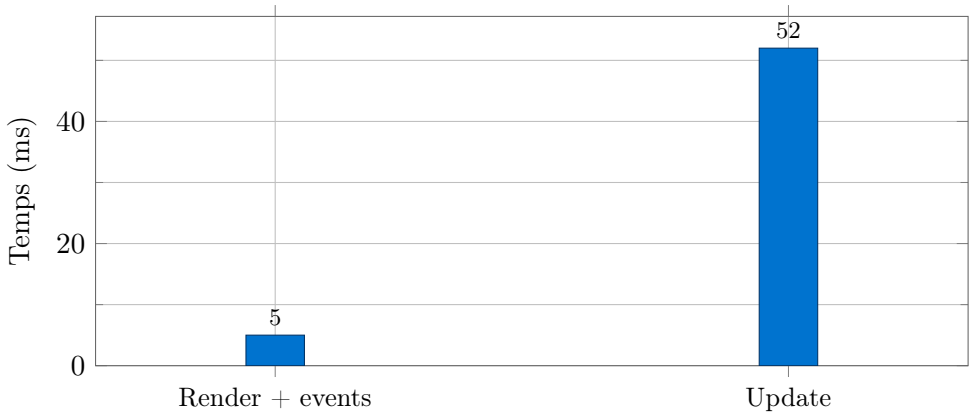


FIGURE 1 – Comparaison des deux temps relevés dans la boucle interactive de référence

5 Étape 1 : parallélisation avec Numba

5.1 Objectif

Cette étape vise à exploiter le parallélisme mémoire partagée disponible sur la machine locale. La machine de test dispose d'un processeur AMD Ryzen 7 5800H avec 8 cœurs physiques et 16 threads logiques. La campagne de mesure utilise $1, 2, 4, 8, 16$ threads.

5.2 Changements clés d'implémentation

- une nouvelle version du script a été créée dans `stages/01_numba_threads/` pour conserver l'étape de référence intacte ;
- les boucles indépendantes ont été parallélisées avec `Numba` et `prange`, en particulier pour le calcul des accélérations, les masses/centres de masse des cellules et l'intégration de Verlet ;
- les parties sensibles aux *race conditions*, comme le comptage et l'insertion des corps dans les cellules, ont été conservées en série ;
- un mode `-benchmark` sans affichage a été ajouté pour mesurer uniquement le coût du calcul.

5.3 Principes de parallélisation retenus

Dans cette étape, les choix retenus sont les suivants :

- paralléliser le calcul des accélérations, car chaque itération met à jour uniquement l'accélération d'un corps ;
- paralléliser le calcul des masses et centres de masse des cellules, car chaque cellule écrit dans un emplacement propre ;
- paralléliser les boucles d'intégration de Verlet sur les corps ;
- conserver en série le comptage des corps par cellule et leur insertion dans les tableaux d'indices, car ces boucles modifient des compteurs partagés et introduiraient des *race conditions*.

5.4 Benchmark sans affichage

Le speed-up de l'étape 1 est mesuré en mode *compute-only*, sans SDL/OpenGL, afin d'isoler le coût de calcul. Cette décision est essentielle pour deux raisons :

- éviter que le rendu graphique ne masque les gains réels du calcul parallèle ;
- exclure la première itération chronométrée à cause du coût de compilation JIT de Numba.

Le script de cette étape propose donc deux modes :

- simulation interactive ;
- benchmark sans affichage avec `-benchmark -steps N -warmup M`.

En mode benchmark, la quantité mesurée est plus précise que dans le visualiseur : chaque temps élémentaire correspond uniquement à un appel à `local_system.update_positions(dt)`. Il s'agit donc du coût du calcul physique pur d'un pas de temps, sans gestion de fenêtre, sans rendu et sans copie des positions vers l'interface graphique.

6 Méthodologie expérimentale de l'étape 1

6.1 Environnement expérimental

Paramètre	**Valeur**
CPU	AMD Ryzen 7 5800H
Cœurs physiques	8
Threads logiques	16
Python	3.12.3
Numba	0.64.0
Jeu de données principal	`data/galaxy_5000`
Pas de temps	0.0015
Grille	$15 \times 15 \times 1$
Warmup benchmark	1 pas
Pas mesurés	30

Table 2 – Environnement et paramètres de mesure pour l'étape 1

Les résultats de cette étape correspondent à la campagne de benchmark exécutée sur `data/galaxy_5000`, avec $dt = 0.0015$, une grille $15 \times 15 \times 1$, un pas de warmup non mesuré et 30 pas mesurés par configuration.

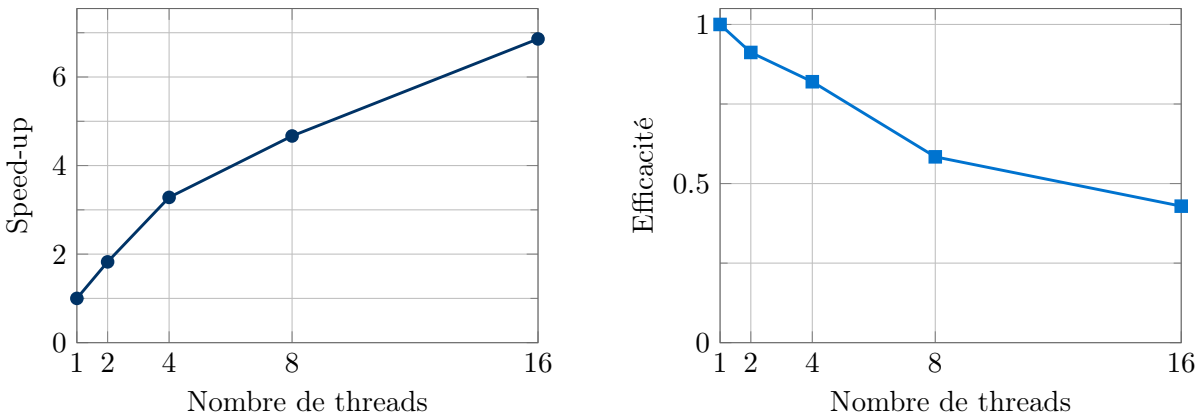
7 Résultats de l'étape 1

7.1 Benchmark compute-only

Threads	Temps moyen par pas (ms)	Speed-up	Efficacité
1	227.970	1.000	1.000
2	124.934	1.825	0.912
4	69.480	3.281	0.820
8	48.834	4.668	0.584
16	33.233	6.861	0.429

Table 3 – Résultats du benchmark compute-only pour l'étape 1

7.2 Graphiques du benchmark compute-only



(a) Speed-up

(b) Efficacité

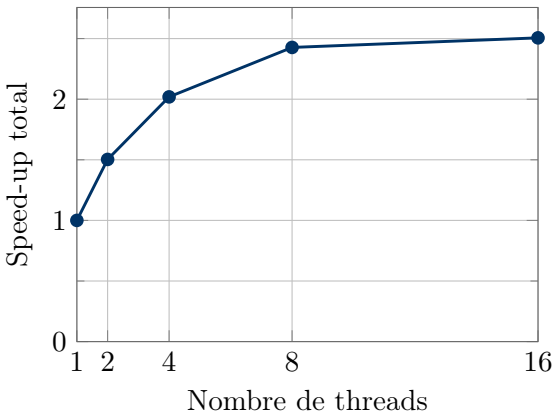
Figure 2 – Scalabilité observée sur le benchmark compute-only de l'étape 1

7.3 Benchmark interactif

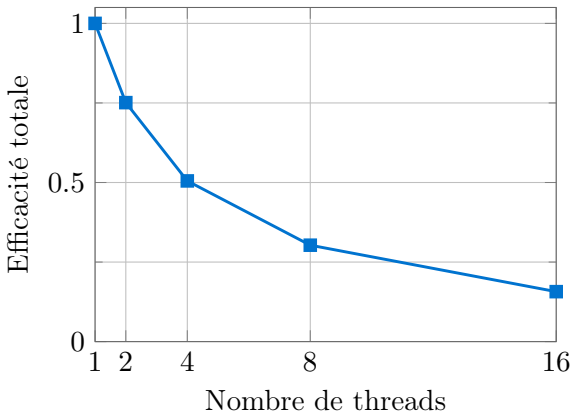
Pour préparer la comparaison avec l'étape 2, on a aussi mesuré l'étape 1 en mode interactif, avec fenêtre ouverte, sur `data/galaxy_1000`. Le temps total correspond ici à `Render time + Update time`.

Threads	Render (ms)	Update (ms)	Total (ms)	Speed-up total	Efficacité totale
1	4.500	16.800	21.300	1.000	1.000
2	5.025	9.150	14.175	1.503	0.751
4	4.750	5.800	10.550	2.019	0.505
8	4.475	4.300	8.775	2.427	0.303
16	4.688	3.812	8.500	2.506	0.157

Table 4 – Résultats du benchmark interactif pour l'étape 1



(a) Speed-up total



(b) Efficacité totale

FIGURE 3 – Graphiques du benchmark interactif pour l'étape 1

On observe une amélioration nette du temps total, mais plus limitée qu'en compute-only. La partie `Render time` reste presque constante, ce qui borne mécaniquement le gain global.

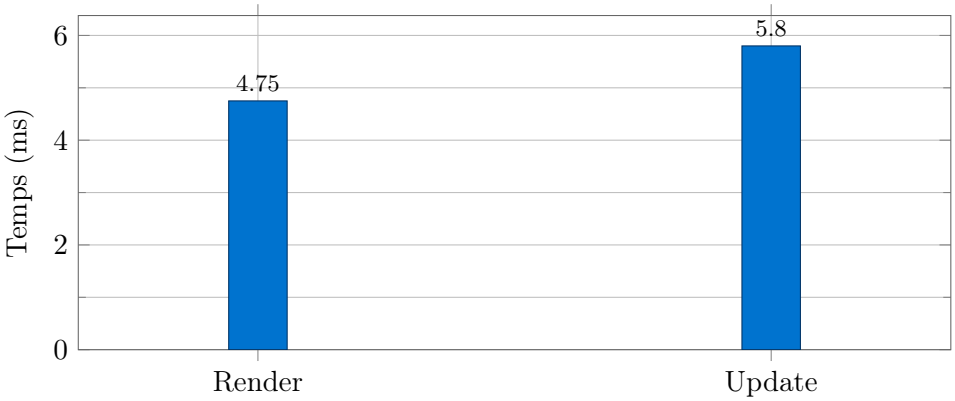


FIGURE 4 – Décomposition des temps interactifs de l'étape 1 à 4 threads

8 Discussion de l'étape 1 avec la loi d'Amdahl

Il faut distinguer deux situations :

- si le benchmark *compute-only* accélère bien, alors la parallélisation Numba du calcul est utile ;
- si la version interactive accélère moins, c'est normal : le rendu et les parties sérielles prennent alors une place plus visible ;
- la comparaison entre 8 et 16 threads permet surtout de voir si ajouter des threads logiques apporte encore un vrai gain.

Les mesures obtenues vont dans ce sens. Le speed-up augmente bien jusqu'à 16 threads, mais l'efficacité baisse progressivement. Le passage de 8 à 16 threads aide encore, mais moins que les premiers paliers. Cela suggère qu'une partie du temps reste limitée par des sections sérielles ou par les accès mémoire.

Un essai interactif sur `data/galaxy_1000` montre aussi qu'après parallélisation le temps de mise à jour tombe autour de 4 ms, donc du même ordre que le rendu graphique. En pratique, cela

veut dire que l'affichage commence à devenir une nouvelle limite.

Le message principal de cette étape est donc simple :

— Numba améliore clairement le temps de calcul ;
— mais cette amélioration ne peut pas se traduire entièrement dans la version interactive ;
— la suite logique est donc de séparer calcul et affichage dans l'étape suivante.

9 Étape 2 : séparation affichage / calcul avec MPI

9.1 Objectif

L'objectif de cette étape est d'isoler le travail graphique du travail numérique. Au lieu d'exécuter rendu et calcul dans le même processus, on utilise ici exactement deux processus MPI :

— le `rank 0` gère uniquement la fenêtre, les événements SDL et le rendu OpenGL ;
— le `rank 1` conserve tout l'état du système physique et exécute `update_positions(dt)`.

Cette organisation prépare naturellement la suite du sujet : avant de distribuer réellement le calcul entre plusieurs processus, on vérifie déjà que l'affichage peut être extrait du chemin critique.

9.2 Changements clés d'implémentation

— un nouveau script MPI a été créé dans `stages/02_mpi_display_compute/`, avec exactement deux processus ;
— le `rank 0` ne calcule plus la physique : il reçoit les positions et met uniquement à jour le visualiseur ;
— le `rank 1` conserve tout l'objet `NBodySystem` et réutilise les noyaux Numba de l'étape 1 pour calculer les pas de temps ;

9.3 Pourquoi exactement deux processus

Cette étape ne cherche pas encore à paralléliser le calcul par domaine spatial. Le but est plus simple : séparer proprement les responsabilités. Deux processus suffisent donc :

— un processus d'affichage ;
— un processus de calcul.

Utiliser plus de processus ici ajouterait de la complexité MPI sans répondre à l'objectif de l'étape.

9.4 Protocole de communication

Le protocole MPI retenu est volontairement minimal.

— Au démarrage, le `rank 1` envoie au `rank 0` les données nécessaires à l'affichage initial : positions, couleurs, intensités lumineuses et bornes de la scène.
— À chaque pas interactif, le `rank 0` poste d'abord un `Irecv` pour le prochain tableau de positions, puis envoie la commande `STEP`.
— Le `rank 1` reçoit `STEP`, calcule le pas suivant, puis envoie les nouvelles positions.
— À la fermeture, le `rank 0` envoie `STOP` et attend un accusé de réception.

Le choix `Irecv` puis `Send` suit directement le schéma vu en cours : le récepteur se déclare prêt avant l'envoi, ce qui évite les interblocages inutiles.

10 Résultats de l'étape 2

10.1 Mode interactif

L'étape 2 doit surtout être lue en mode interactif. Sur `data/galaxy_1000`, avec `mpirun -bind-to none -np 2`, le temps total par image correspond ici à `Render time + MPI wait + point update time`.

Threads	Render (ms)	MPI wait + update (ms)	Total (ms)	Speed-up total	Efficacité totale
1	5.450	18.125	23.575	1.000	1.000
2	4.350	8.825	13.175	1.789	0.895
4	4.425	6.025	10.450	2.256	0.564
8	4.825	4.800	9.625	2.449	0.306
16	4.500	3.800	8.300	2.840	0.178

Table 5 – Résultats du mode interactif pour l'étape 2

On voit donc que le `rank 0` passe une partie du temps à attendre la nouvelle trame calculée par le `rank 1`, mais que le temps total reste du même ordre que celui de l'étape 1. La séparation MPI ne dégrade donc pas brutalement l'usage interactif.

10.2 Benchmark MPI hors affichage

Le tableau suivant ne représente pas le mode interactif. Il s'agit d'un benchmark MPI sans fenêtre, utilisé seulement pour estimer proprement le coût du calcul du `rank 1` et le surcoût de communication.

Threads	rank1_compute (ms)	end-to-end (ms)	Surcoût MPI (ms)	Speed-up	Efficacité
1	246.802	246.919	0.117	1.000	1.000
2	138.732	138.853	0.121	1.778	0.889
4	79.649	79.767	0.118	3.096	0.774
8	52.546	52.673	0.127	4.688	0.586
16	42.668	42.801	0.132	5.769	0.361

Table 6 – Résultats du benchmark MPI sans affichage

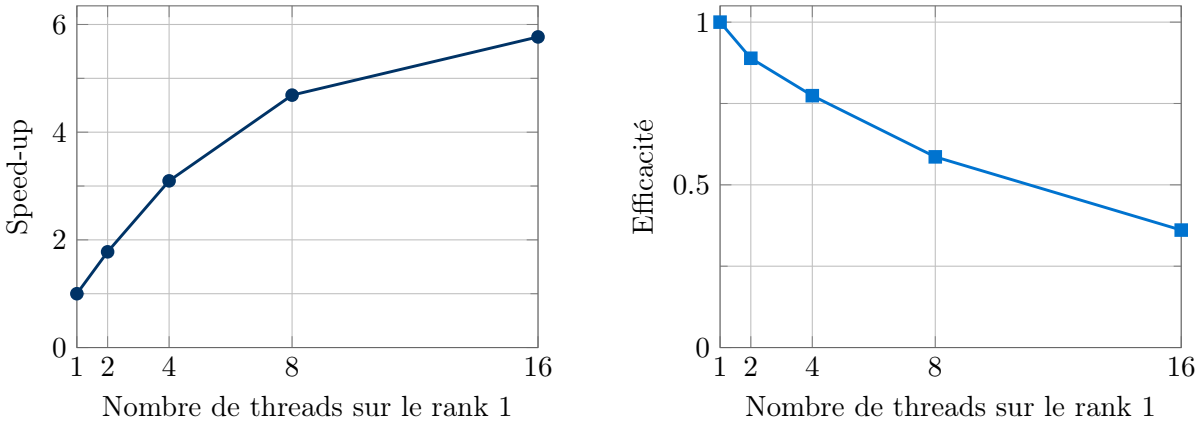
10.3 Lecture directe du benchmark

Deux points ressortent immédiatement :

- le temps `end-to-end` suit presque exactement le temps de calcul du `rank 1` ;
- le surcoût moyen de communication reste très faible, proche de 0.12 ms par pas sur toute la campagne.

Autrement dit, la séparation MPI est fonctionnelle et son coût de transport reste négligeable à cette échelle. Le temps total du benchmark reste donc dominé par le calcul gravitationnel lui-même.

10.4 Graphiques du benchmark



(a) Speed-up bout-en-bout

(b) Efficacité bout-en-bout

FIGURE 5 – Scalabilité observée sur le benchmark MPI hors affichage de l'étape 2

11 Comparaison entre l'étape 1 et l'étape 2

11.1 Comparaison interactive

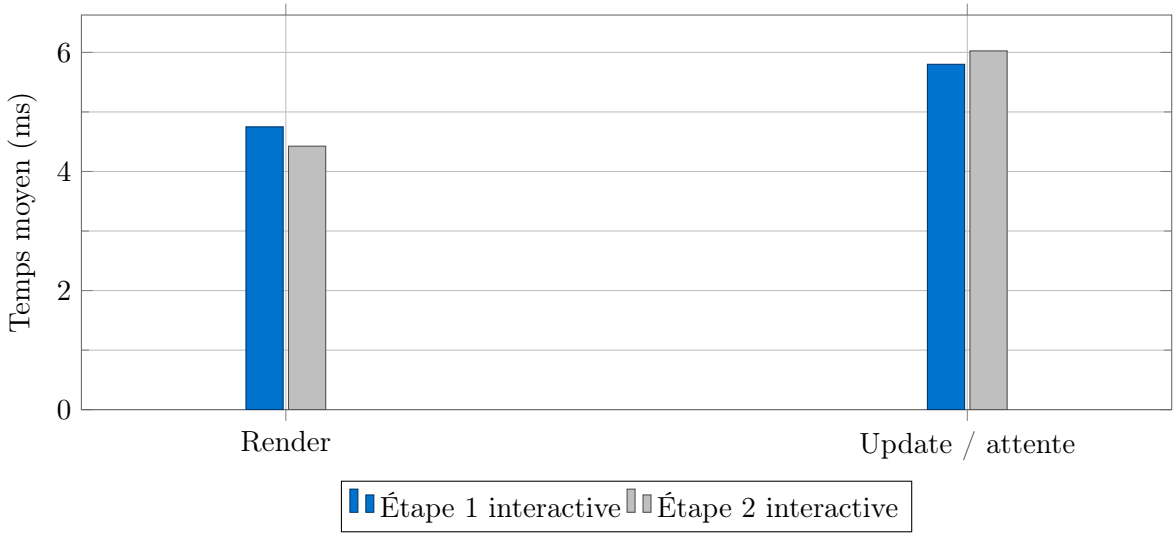


FIGURE 6 – Décomposition des temps interactifs à 4 threads

À 4 threads, les deux étapes ont pratiquement le même coût interactif total : 10.550 ms pour l'étape 1 contre 10.450 ms pour l'étape 2. L'étape 2 dépense un peu plus de temps en attente MPI, mais elle compense par un rendu légèrement plus court.

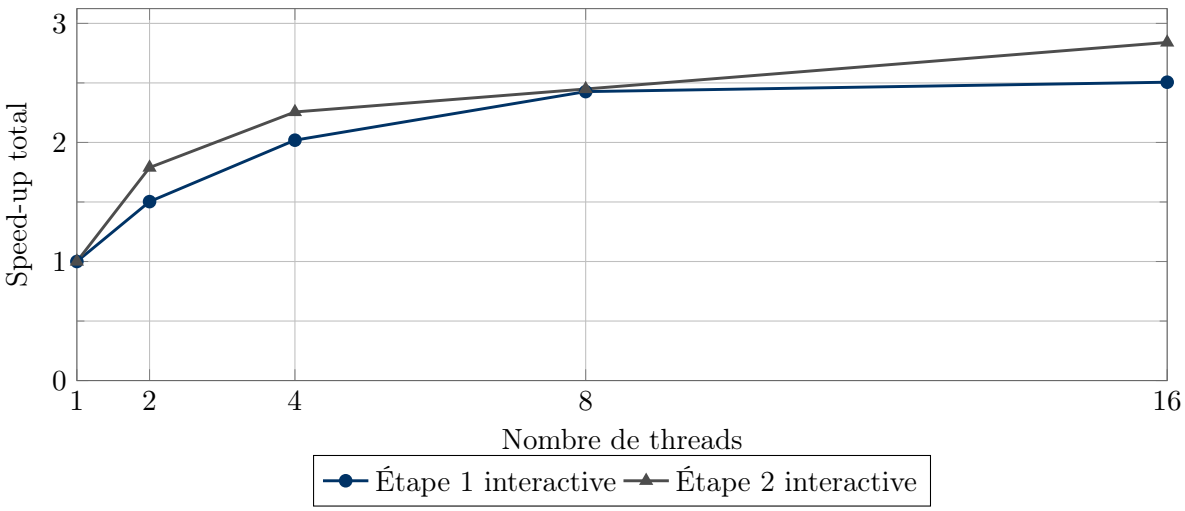


FIGURE 7 – Comparaison des speed-up interactifs totaux entre l'étape 1 et l'étape 2

Le résultat le plus utile pour la suite est que l'étape 2 scale légèrement mieux en mode interactif. Le gain reste modéré, mais il existe : à 16 threads, on obtient un speed-up total de 2.840 contre 2.506 pour l'étape 1. Autrement dit, la séparation affichage/calcul n'apporte pas un saut massif immédiat, mais elle isole correctement le rendu sans pénaliser le coût par image, ce qui prépare proprement l'étape suivante.

12 Étape 3 : calcul MPI complet avec cellules fantômes

12.1 Objectif

L'objectif de cette étape est de distribuer réellement le calcul sur plusieurs processus MPI. Le `rank 0` reste dédié à l'affichage, mais les `ranks 1..P-1` se partagent maintenant la grille de calcul et ne conservent chacun qu'une partie des particules.

12.2 Changements clés d'implémentation

- le `rank 0` lit le fichier initial une seule fois, crée un `global_id` fixe pour chaque étoile et distribue les particules aux workers ;
- les workers forment une grille 2D logique sur le plan Oxy, avec une décomposition en blocs rectangulaires de cellules ;
- chaque worker stocke ses particules propriétaires, plus des copies fantômes nécessaires aux interactions directes proches ;
- après chaque pas, les positions sont réunies sur le `rank 0` par `global_id` pour reconstruire l'ordre global avant affichage.

12.3 Topologie retenue

Conformément au cours sur la décomposition de domaine, on ne découpe pas l'espace en bandes 1D. Les workers sont organisés en une grille 2D $p_x \times p_y$ sur Ox et Oy, tandis que l'axe Oz reste entier sur chaque worker.

Cette étape n'accepte donc que les exécutions où le nombre de workers se factorise en une vraie grille 2D, avec $p_x > 1$ et $p_y > 1$. Le cas à deux processus reste celui de l'étape 2.

12.4 Rôle des cellules fantômes et des réductions globales

Le noyau gravitationnel utilise un calcul direct dans les cellules voisines situées à distance de Chebyshev inférieure ou égale à 2. C'est la raison pour laquelle le halo MPI est fixé à deux cellules dans x et y, diagonales incluses.

Pour les cellules lointaines, chaque worker n'utilise pas les particules une à une, mais la masse totale et le centre de masse globaux de chaque cellule. Ces quantités sont reconstruites par `Allreduce` en sommant uniquement les particules propriétaires. Les particules fantômes ne servent donc qu'aux interactions proches et ne participent jamais au comptage global.

13 Résultats de l'étape 3

13.1 Campagne de mesures

La campagne officielle a été menée sur `data/galaxy_5000`, avec $dt = 0.0015$, une grille $15 \times 15 \times 1$, un pas de warmup et 30 pas mesurés. Les couples (workers, threads) retenus respectent la contrainte workers $\times$ threads ≤ 16 sur cette machine.

Pour les cas à 9 processus MPI au total, il a fallu ajouter l'option `-use-hwthread-cpus` à `mpirun`, car OpenMPI compte par défaut les slots sur les seuls cœurs physiques.

13.2 Table 2D des temps bout-en-bout

Workers MPI	1 thread	2 threads	4 threads
4	125.816	72.342	52.114
6	163.798	100.596	N/A
8	139.213	Instable	N/A

Table 7 – Temps bout-en-bout moyens de l'étape 3, en ms

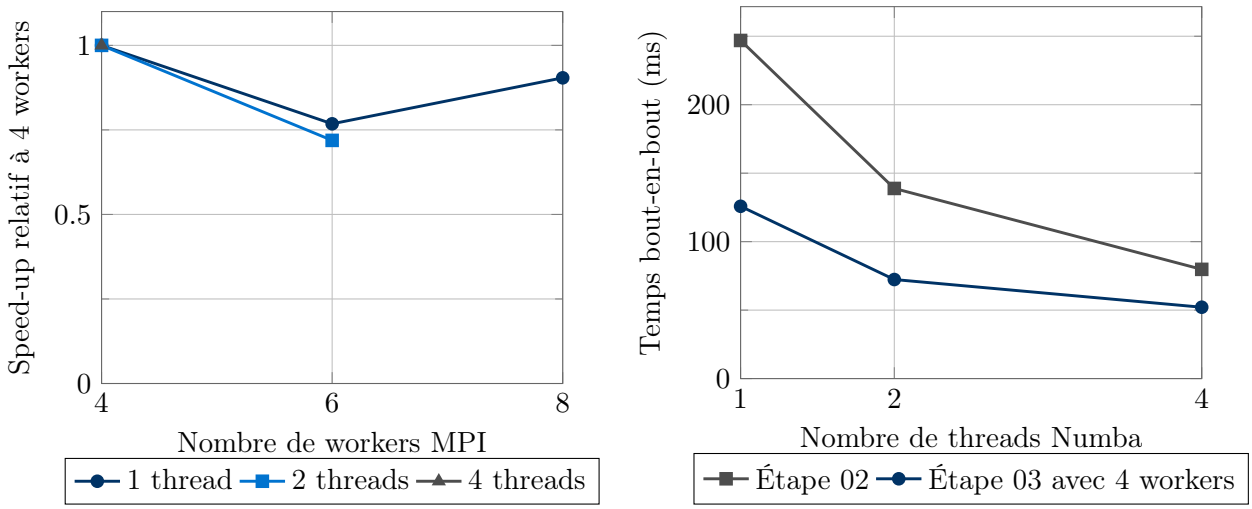
Dans cette table, `N/A` signifie qu'une configuration n'a pas été mesurée parce qu'elle dépasserait la limite workers $\times$ threads ≤ 16. Le terme « Instable » désigne au contraire une configuration réellement testée, mais écartée car elle produit des valeurs non finies au cours du calcul.

13.3 Détails sur les configurations stables

Workers	Threads	end-to-end (ms)	worker mean (ms)	worker max (ms)	Migration (ms)	Halo (ms)	Réduction (ms)
4	1	125.816	101.738	125.148	26.652	0.493	0.255
4	2	72.342	58.431	71.717	12.227	0.488	0.259
4	4	52.114	43.161	51.457	7.620	0.537	0.276
6	1	163.798	108.963	162.997	52.802	0.596	0.325
6	2	100.596	68.752	99.907	32.172	0.627	0.347
8	1	139.213	95.797	138.461	47.931	0.658	0.336

Table 8 – Mesures détaillées des configurations stables de l'étape 3

13.4 Graphiques



(a) Speed-up relatif à 4 workers

(b) Étape 2 vs étape 3 à nombre de threads égal

FIGURE 8 – Lecture globale des performances de l'étape 3

Le résultat principal est simple : l'étape 3 devient plus rapide que l'étape 2 à nombre de threads égal dès que l'on utilise 4 workers. Par exemple, à 4 threads, le temps bout-en-bout passe de 79.767 ms à l'étape 2 à 52.114 ms à l'étape 3.

Le gain reste net sur les trois configurations stables à 4 workers : environ $\times 1.96$ à 1 thread, $\times 1.92$ à 2 threads et $\times 1.53$ à 4 threads par rapport à l'étape 2.

En revanche, augmenter encore le nombre de workers ne donne pas ici un meilleur speed-up. Les configurations à 6 et 8 workers deviennent moins bonnes que celle à 4 workers, ce qui montre que la décomposition de domaine ne suffit pas à elle seule : le coût de migration et surtout le déséquilibre entre workers prennent alors le dessus.

14 Déséquilibre de charge et splitting adaptatif

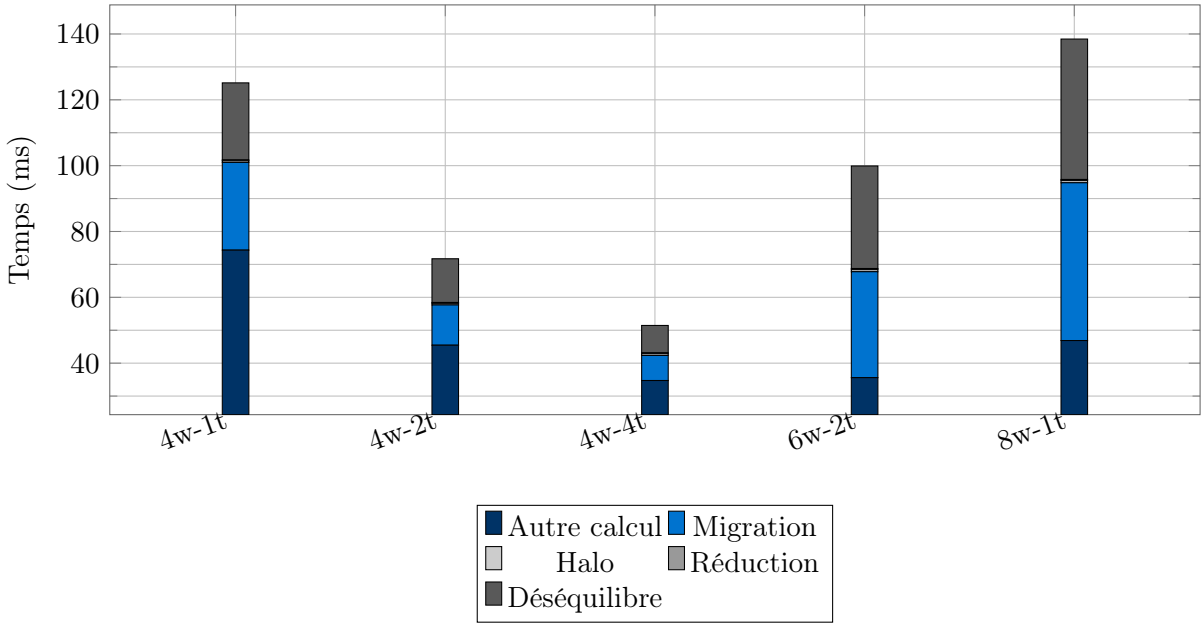


FIGURE 9 – Décomposition du temps critique sur plusieurs configurations stables de l'étape 3

La figure précédente montre que le halo et la réduction globale restent très faibles. Le vrai problème n'est donc pas la communication de voisinage elle-même. Deux termes dominent beaucoup plus vite :

— la migration des particules entre sous-domaines ;
— l'écart entre le worker moyen et le worker le plus lent.

Cette perte de balance vient de la densité non uniforme de la galaxie. Le centre contient beaucoup plus d'étoiles que la périphérie. Avec un découpage régulier en cellules, certains workers héritent d'une zone très dense et deviennent le chemin critique, tandis que d'autres finissent plus tôt et attendent.

Le cours suggère alors une idée naturelle : utiliser un *splitting adaptatif*. Le principe serait de donner des sous-domaines plus petits aux zones denses et plus grands aux zones peu peuplées, afin de mieux répartir le nombre réel de particules et non seulement le nombre de cellules.

Cette solution améliorerait l'équilibre de charge, mais elle introduirait un nouveau coût :

— interfaces plus irrégulières ;
— gestion plus complexe des voisins ;
— migration plus difficile à maintenir ;
— halos moins réguliers et donc communication plus délicate.

Autrement dit, l'étape 3 valide bien la décomposition de domaine et les cellules fantômes, mais elle montre aussi pourquoi une grille régulière ne suffit pas à obtenir une bonne scalabilité sur une galaxie très concentrée.

15 Conclusion intermédiaire

Les trois premières étapes installent désormais une base expérimentale cohérente pour la suite du projet :

— une implémentation autonome de l'étape 1 dans `stages/01_numba_threads/`, dédiée au calcul partagé avec Numba ;

— une implémentation autonome de l'étape 2 dans `stages/02_mpi_display_compute/`, dédiée à la séparation MPI entre affichage et calcul ;
— une implémentation autonome de l'étape 3 dans `stages/03_mpi_full_ghost/`, dédiée à la distribution réelle du calcul sur plusieurs workers MPI ;
— une stratégie de benchmark reproductible distinguant calcul local, coût bout-en-bout et principaux surcoûts de communication ;

L'étape 2 montre d'abord qu'il est possible de séparer proprement affichage et calcul avec MPI tout en gardant un surcoût de transfert très faible, de l'ordre de 0.12 ms par pas seulement dans la configuration retenue.

L'étape 3 valide ensuite la décomposition de domaine 2D avec cellules fantômes. Sur cette machine, la meilleure configuration mesurée est celle à 4 workers : elle devient plus rapide que l'étape 2 à nombre de threads identique, avec par exemple 52.114 ms contre 79.767 ms à 4 threads.

La limite principale apparaît quand on augmente encore le nombre de workers. Le coût des halos reste faible, mais la migration des particules et surtout le déséquilibre de charge deviennent dominants. Le centre galactique concentre trop de particules pour qu'un découpage régulier reste efficace. La suite logique sera donc d'analyser comment un splitting adaptatif, ou une structure hiérarchique de type Barnes-Hut, pourrait mieux répartir la charge sans faire exploser la complexité des communications.